

Project Report - Transaction network analysis

Ilija Chrchev

89241041@student.upr.si

University of Primorska, FAMNIT

Koper, Slovenia

ABSTRACT

This report presents a system for detecting wash trading in Ethereum NFT markets through linkability network analysis. Wash trading is a form of market manipulation in which traders artificially inflate the volume or price of an asset by trading it back and forth between accounts they secretly control. To uncover such behavior, we build a linkability network by performing a bounded breadth first search(BFS) on the Ethereum Transaction Network and connecting any two traders that share a path of transactions within a fixed depth.

The same algorithm is implemented in three execution modes, a **sequential** version, a **multithreaded parallel** version and a **distributed** version based on MPI. The report describes the design of each version, the classes and methods I am using, as well as the problems I encountered along the way of building, together with how each problem was approached and solved.

KEYWORDS

NFT, wash trading, Ethereum, BFS, parallel computing, MPI

1 INTRODUCTION

Non-fungible tokens known as NFTs are digital assets traded on public blockchains like Ethereum. Because every transaction is recorded and visible, these markets look transparent, but this transparency is often abused through *wash trading*, where the same person controls both sides of a trade to inflate volume and prices. Spotting wash trading is hard because Ethereum addresses are anonymous and one person can own many wallets. What we can see is how money moves between them. If two NFT traders are connected by a short chain of transactions, they are likely linked. This is the idea behind *linkability network analysis*.

2 PROBLEM STATEMENT

Given the Ethereum Transaction Network and the set of traders extracted from the Bored Ape Yacht Club (BAYC) collection, the goal is to find all pairs of traders that are connected by a path of transactions of length at most k , while ignoring paths that pass thorough blacklisted addresses such as exchanges and other high traffic services that almost everyone interacts with. The output is a linkability network, a graph where each node is a BAYC trader and each edge marks a pair of traders that are linkable within depth k . The challenge is doing this efficiently on a dataset of around 5 GB, which is what motivates the three execution modes presented in the next sections.

3 SHARED CLASSES

Before describing the three implementations, it is worth presenting the classes that all of them rely on. These classes handle loading

the data, representing the graph and providing the helpers used by the BFS, regardless of execution mode. They live in the `model`, `builder` and `utils` packages.

3.1 AddressMapper

Ethereum addresses are 42 character hexadecimal strings, which are expensive to store and compare. `AddressMapper` assigns each unique address an integer ID the first time it is seen, so the rest of the system can work with primitive `int` values instead of strings. This single change drastically reduces memory usage and speeds up every lookup in the graph.

Main methods:

- `getOrCreateId(String address)` returns the ID of an address, creating a new one if it has not been seen before.
- `getAddress(int id)` returns the original address for a given ID.
- `size()` returns the total number of mapped addresses, which is also the number of nodes in the graph.
- `clear()` releases the internal maps once they are no longer needed, to free memory before the BFS starts.

3.2 Graph

`Graph` stores the Ethereum Transaction Network as a map from each node to its list of neighbours, where the neighbour list is an `IntVec`, a compact wrapper around a primitive `int[]` array. This is much faster and more memory efficient than using boxed collections, since the BFS only ever needs to iterate over a node's neighbours.

Main methods:

- `addEdge(int from, int to)` adds a directed edge between two nodes.
- `getConnected(int nodeId)` returns the `IntVec` of neighbours of a node.
- `duplicateEdges()` sorts each neighbour list and removes duplicate entries that come from repeated transactions between the same two addresses.
- `nodeCount()` and `edgeCount()` return the size of the graph.

3.3 GraphBuilder

`GraphBuilder` reads the ETN CSV file line by line and builds the Graph together with the `AddressMapper`. For each transaction it extracts the from and to addresses, skips the row if either of them is blacklisted, maps the addresses to integer IDs and adds the edge to the graph. Self loops are skipped.

Main methods:

- `buildFromETN(String etnFilePath)` performs the full read of the CSV file and populates the graph.

- `isBlackListed(String address)` checks whether a given address is in the blacklist.
- `getGraph()` and `getAddressMapper()` return the populated structures so they can be used by the next stages.

3.4 NFTTraderLoader

NFTTraderLoader reads the BAYC CSV file and produces the set of trader IDs that the BFS will run from. For each transfer it considers both the sender and the receiver, skipping the null address (which represents minting and burning) and any blacklisted address. The remaining addresses are mapped to integer IDs through the same AddressMapper used for the graph, so traders and graph nodes share the same ID space.

Main methods:

- `loadTraders(String nftFilePath)` reads the file and returns the set of trader IDs.
- `isBlackListed(String address)` same blacklist check as in GraphBuilder.

3.5 BlacklistReader

BlacklistReader loads all blacklist JSON files from a folder and merges them into a single `Set<String>`. The blacklist contains hub addresses that should be ignored during graph building and trader loading, since they would otherwise produce false connections between unrelated traders.

Main method:

- `loadBlacklist(String folderPath)` reads every .json file in the folder, extracts every string starting with 0x and returns the merged set.

3.6 Logger

Logger is a small utility that prints timestamped log messages to the console at different levels: info, warn, error, debug and success. Each level is printed in a different color so that progress, warnings and errors are easy to distinguish during long runs.

4 SEQUENTIAL IMPLEMENTATION

The sequential implementation is the baseline version of the system. It runs the entire pipeline on a single thread: load the blacklist, build the graph, load the traders, then run BFS from each trader one after another. The entry point is Main, and the BFS itself is encapsulated in the LinkabilityBuilder class.

4.1 SequentialMain

Main ties all the shared classes together and runs them in order. It reads the command line arguments (max depth, ETN file, NFT file, blacklist folder, output file) or falls back to default values, then executes the pipeline:

- (1) Load the blacklist with BlacklistReader.
- (2) Build the ETN graph with GraphBuilder, then call `duplicateEdges()` on the graph to sort and deduplicate neighbour lists.
- (3) Load the BAYC traders with NFTTraderLoader.
- (4) Run `LinkabilityBuilder.buildLinkabilityNetwork()` to perform the BFS and write the output CSV.

Each stage is timed individually and the total runtime is reported at the end.

4.2 LinkabilityBuilder

LinkabilityBuilder is the heart of the sequential version. It holds a reference to the graph, the set of trader IDs, the maximum depth and the node count, and exposes a single public method, `buildLinkabilityNetwork(String outputFile)`, which runs a bounded BFS from every trader and writes one CSV line for each linkable pair found.

The method works as follows. First, the trader set is converted into a boolean lookup array `isTrader[]` of size `nodeCount`, so checking whether a node is a trader becomes a single array access during the BFS. Three primitive arrays are allocated once and reused across all BFS runs: `seen[]` for visited markers, `distance[]` for BFS levels, and `queue[]` as the BFS frontier.

For each source trader, a fresh BFS is run by simply incrementing a counter `bfsId` and using it as the visited marker. A node is considered visited in the current BFS only if `seen[node] == bfsId`.

During the traversal, when a neighbour is reached at distance `n`, the BFS checks whether that neighbour is itself a trader. If it is and it is not the source, a line `source,neighbour,n` is written to the output file and the link counters are updated.

The queue is stored as a primitive `int[]` that grows on demand through the helper method `grow()`. The counter `bfsId` is reset back to one and the whole `seen[]` array is cleared in the rare case it would overflow `Integer.MAX_VALUE`, which in practice never triggers but is there for safety.

Main methods:

- `buildLinkabilityNetwork(String outputFile)` runs the full BFS over all traders and writes the resulting linkability network.
- `grow(int[] array)` doubles the size of the BFS queue when needed.

4.3 Challenges Faced

A few problems came up during the development of the sequential version.

The first was raw scale. The ETN CSV is around 5 GB and contains millions of lines, so any approach that allocates per row objects or builds intermediate structures runs out of memory before it finishes. The fix was to read the file with a `BufferedReader` line by line and to push everything into the integer graph immediately, without keeping the original strings around.

The second was duplicate edges. The same pair of addresses can transact many times, which means the same edge gets added repeatedly to the adjacency list. This made the BFS slower without changing its result. The `duplicateEdges()` method on Graph solves this by sorting each neighbour list and removing repeated entries.

The third was a subtle bug related to sizing the BFS arrays. The first version used `graph.nodeCount()` to size `seen[]` and `distance[]`, but `nodeCount()` only counts nodes that have outgoing edges. Some addresses appear only as edge targets, never as sources, and they have integer IDs that fall outside that range.

This caused an `ArrayIndexOutOfBoundsException` when the BFS reached such a node. The fix was to size all BFS arrays based on `addressMapper.size()` instead, which counts every mapped address.

5 PARALLEL IMPLEMENTATION

The parallel implementation keeps the same algorithm as the sequential one but runs the BFS for many traders at the same time. Since each BFS starts from a different source and only reads from the graph, the traders can be processed independently as long as the workers do not interfere with each other through shared state. The entry point is `ParallelMain`, and the BFS logic lives in `ParallelLinkabilityBuilder`.

5.1 ParallelMain

`ParallelMain` performs the same pipeline as the sequential `Main`: load the blacklist, build the graph, deduplicate edges, load the traders. The only difference is the last step, where it constructs a `ParallelLinkabilityBuilder` instead of a `LinkabilityBuilder`. It also logs the number of available processors at the start so the run is reproducible.

5.2 ParallelLinkabilityBuilder

`ParallelLinkabilityBuilder` runs the BFS in parallel using a fixed thread pool of size equal to the number of available processors. Each trader becomes one task submitted to the pool. To make this safe and fast, three concerns are handled separately: per thread state, shared counters and concurrent file output.

Per thread state. Each BFS needs three big arrays: `seen[]`, `distance[]` and `queue[]`. Allocating them per task would dominate the runtime, so instead they are wrapped in a private `BfsWorkspace` class and stored in a `ThreadLocal`. The first time a worker thread runs a task it allocates one workspace, and every following task on that thread reuses it. The same BFS stamping trick from the sequential version is used here through `nextBfsId()`, so even though `seen[]` is reused across thousands of BFS runs, no resetting is needed between them.

Shared counters. The total number of links and the per weight breakdown are written to from every thread, so they are stored as `AtomicLong` and `AtomicLong[]` respectively. To keep contention low, each task first accumulates its own counts into local variables (`localLinks` and `localLinksByWeight[]`) and only adds them to the shared atomics once at the end of the BFS.

Concurrent file output. Multiple threads cannot write to the same file safely, and using a lock would slow the workers down. Instead, each worker builds its output into a local `StringBuilder` during the BFS and puts the whole string onto a shared queue at the end. A single writer thread pulls chunks off the queue and writes them to the file, so only one thread ever touches the disk.

When all tasks are done, the main thread shuts down the pool and the writer thread shutdowns the signal, finishes flushing whatever it has and exits. The main thread joins on it before returning.

Main methods and inner class:

- `buildLinkabilityNetwork(String outputFile)` starts the writer thread, submits one BFS task per trader, waits for completion and shuts everything down cleanly.
- `BfsWorkspace` is a private inner class that holds the per thread `seen[]`, `distance[]` and `queue[]` arrays plus the current `bfsId`. It exposes `nextBfsId()` and `ensureQueueCapacity()` as helpers.

5.3 Challenges Faced

The first challenge was handling the visited arrays. `ThreadLocal` solved it, but it required moving the BFS state out of the builder and into a small inner class that lives with a thread, not with a task.

The second was safe file output. Locking the writer worked but was slow, so I switched to a dedicated writer thread with a shared queue. The poison pill was added so the writer knows when to stop.

The third was contention on the atomic counters. Updating them from inside the inner loop was correct but slow. Moving to local counters with one `addAndGet()` at the end fixed it.

The fourth was the shutdown order. `pool.awaitTermination()` has to finish before the poison pill is sent, otherwise the last few results would be lost.

6 DISTRIBUTED IMPLEMENTATION

The distributed implementation runs the same BFS across multiple processes using MPI. Each process loads the full graph and trader set, then handles only its own share of the traders. The entry point is `DistributedMain`, and the BFS logic lives in `DistributedLinkabilityBuilder`. The project uses MPJ Express 0.44 in multi-core mode.

6.1 DistributedMain

`DistributedMain` runs on every rank in parallel. It starts with `MPI.Init(args)` and reads the rank and total size from `MPI.COMM_WORLD`. Only rank 0 loads the blacklist, builds the graph, deduplicates edges and loads the traders. It then sends the graph and the trader set to the other ranks through `MPI.COMM_WORLD.Bcast()`. This is still a **replicated graph** design, since every rank ends up with the full ETN graph in memory, but the file is read only once instead of once per rank.

To send the graph efficiently, rank 0 flattens the adjacency lists into two `int[]` arrays: one with the start index of each node's neighbours, and one with all neighbours in order. The other ranks receive these arrays and rebuild the same graph locally. The trader set is converted to a sorted `int[]` before being broadcast, so that every rank sees the same order. This is important: the BFS work is split by index, and if the order differed between ranks, two ranks could end up working on the same trader or skipping one.

Before the BFS starts, all ranks synchronize with `MPI.COMM_WORLD.Barrier()` and timing starts from `MPI.Wtime()`. Each rank constructs its own `DistributedLinkabilityBuilder` and calls `buildLocalPart()` to write its share of the output to a temporary file named `output_distributed.csv.rankN.tmp`. After another barrier, the local link counts are combined with `MPI.Reduce(..., MPI.SUM, 0)` so rank 0 ends up with the global totals.

Finally, rank 0 merges the temporary files into a single output CSV by concatenating them in rank order, then logs the totals and the runtime. `MPI.Finalize()` ends the program.

6.2 DistributedLinkabilityBuilder

`DistributedLinkabilityBuilder` is almost identical to the sequential `LinkabilityBuilder`, with two differences: it works on a slice of the trader array instead of the full set, and it writes to a rank-specific temporary file.

The slice is selected with interleaved partitioning:

```
for (int index = rank;
    index < allTraders.length;
    index += size) {
    int source = allTraders[index];
    ...
}
```

With 4 ranks, rank 0 processes traders at indices 0, 4, 8, 12..., rank 1 takes 1, 5, 9, 13..., and so on. This is simpler than splitting the array into contiguous blocks and gives a more balanced workload, since BFS cost varies a lot from one trader to the next and contiguous blocks could land mostly heavy or mostly light traders on the same rank.

The BFS itself is the same as in the sequential version. Each rank has its own `seen[]`, `distance[]` and `queue[]` arrays, uses BFS stamping, and writes one CSV line per link found. The output goes to a per-rank temporary file rather than a shared one, so there is no need for locks or producer-consumer queues between ranks.

Only rank 0 writes the CSV header at the start of its temporary file. The other ranks write only data lines, so when the files are concatenated the header appears exactly once.

Main methods:

- `buildLocalPart(String tempFile)` runs the BFS over the rank's share of traders and returns its local link-by-weight counts.
- `grow(int[] array)` doubles the BFS queue, same helper as in the sequential version.

6.3 Approach

The BFS only reads from the graph, so the traders can be split across ranks without any need for communication during the BFS itself. The only real work is dividing the traders and putting the output back together at the end.

A replicated graph was used instead of a partitioned one because the dataset fits in memory. Splitting the graph between ranks would force them to send messages every time a BFS crossed a boundary, which would be much slower than just giving every rank its own copy.

Traders are split using interleaved partitioning, where rank 0 takes index 0, 4, 8..., rank 1 takes 1, 5, 9..., and so on. This was chosen because traders take very different amounts of time to process. A trader with few neighbours finishes fast, while one near a hub takes much longer. Block splitting could leave one rank doing all the heavy work, while interleaving spreads heavy and light traders evenly.

Each rank writes its results to its own temporary file. Rank 0 merges them at the end. This way no rank has to wait for another while writing.

6.4 Challenges Faced

The first challenge was making the split deterministic. The trader set is a `Set<Integer>`, which has no fixed order, so two ranks could disagree on which trader is at index 5. Converting the set to a sorted `int[]` before splitting fixed this.

The second was timing. `System.currentTimeMillis()` is not reliable across processes, so I used `MPI.Wtime()` with a `Barrier()` on both sides to measure how long the slowest rank took.

The third was the CSV header. If every rank wrote the `from,to,weight` line, the merged file would have it four times. Only rank 0 writes the header now.

The fourth was memory. Every rank holds the full graph and address mapper, so with four ranks on one machine the heap could fill up. Clearing the address mapper and the blacklist after the trader set is built freed enough memory to run cleanly.

7 EXPERIMENTAL SETUP

All experiments were run on a single machine with an AMD Ryzen 7 7840HS (8 cores, 16 threads) and 16 GB of RAM, running Java 21. The dataset is the full Ethereum Transaction Network CSV at around 5 GB, with roughly 5 million nodes and 6 million edges after blacklist filtering. The trader set comes from the Bored Ape Yacht Club transfers and contains 19,714 unique addresses.

The parallel version uses all 16 hardware threads automatically through `Runtime.getRuntime().availableProcessors()`. The distributed version uses MPJ Express 0.44 in multicore mode, where each rank is a separate thread inside one JVM process on the same machine. Four rank counts were tested: `-np 1`, `-np 2`, `-np 4` and `-np 8`.

All three modes produced identical link counts at every depth, confirming that the parallel and distributed versions are correct.

8 RESULTS

8.1 Runtime Comparison

Table 1 shows the total runtime of each mode at three different depths. Depth 3 represents a shallow BFS that finds only short-range links, depth 5 is a medium case, and depth 9 is the deepest run used in this report.

Table 1: Total runtime by mode and depth

Mode	Depth 3	Depth 5	Depth 9
Sequential	41.6 s	53.6 s	309.7 s
Parallel (16 threads)	44.6 s	45.3 s	73.2 s
Distributed (-np 1)	43.7 s	66.6 s	344.5 s
Distributed (-np 2)	46.1 s	59.6 s	182.9 s
Distributed (-np 4)	42.9 s	51.5 s	147.4 s
Distributed (-np 8)	49.5 s	56.0 s	99.5 s

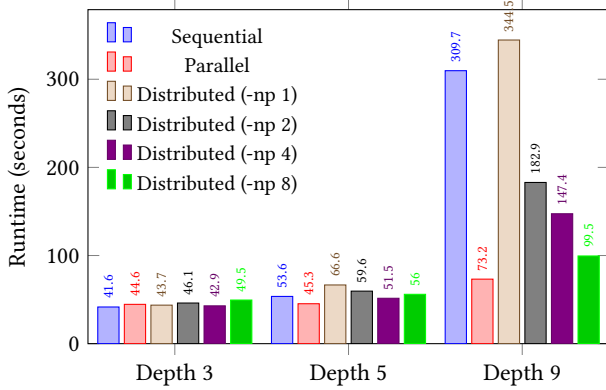


Figure 1: Total runtime by mode and depth.

The parallel version is the fastest mode at every depth, with the largest gap at depth 9 where it finishes in 73.2 seconds against 309.7 seconds for the sequential version, a 4.2× speedup. The advantage grows with depth because deeper BFS spends more time inside the inner loop, where parallelism pays off.

The distributed version shows clear scaling as the number of ranks increases. At depth 9 the total runtime drops from 344.5 seconds with one rank to 99.5 seconds with eight ranks, a 3.5× improvement. At shallow depths the BFS is already so fast that adding ranks does not help, since the fixed loading cost dominates the runtime.

8.2 Where the Time Goes

The total runtime includes both the BFS itself and the one-time setup: reading the 5 GB CSV, building the graph, deduplicating edges, and loading the trader set. The setup is sequential in all three modes, so its cost is constant.

The distributed numbers in Table 1 hide this. The BFS phase of the distributed mode scales well: at depth 9, the BFS itself drops from 297.9 seconds with one rank to 57.5 seconds with eight ranks,

a 5.18× speedup. The reason the total runtime improves by less is that rank 0 reads the 5 GB file and broadcasts the graph to the other ranks before the BFS can start, which costs around 40 seconds regardless of the number of ranks. On a real cluster, each node would load its share of the data in parallel from its own storage, and this setup cost would not grow with the number of ranks.

8.3 Observations

A few clear patterns come out of these numbers.

First, parallel mode is the right choice for a single machine. All threads share one in-memory graph, so there is no duplication, and the BFS work splits cleanly across cores.

Second, distributed mode shows real scaling on the BFS phase, with a 5.18× speedup from one rank to eight ranks at depth 9. The reason the total runtime improves by less is that the broadcast of the graph from rank 0 is serial on this single-machine setup. On a real cluster, this loading would happen in parallel on each node, and distributed mode would outperform the parallel one.

Third, the gap between sequential and parallel widens with depth. At depth 3 the two are nearly equal, since the BFS is short and setup dominates. At depth 9 the BFS dominates and parallelism wins.

9 CONCLUSION

This report presented a system for detecting wash trading in Ethereum NFT markets through linkability network analysis. The core algorithm is a bounded BFS from each BAYC trader over the full Ethereum Transaction Network, with hub addresses filtered out by a blacklist. The same algorithm was implemented in three modes: sequential, multithreaded parallel, and distributed using MPI.

The sequential version was tuned to be fast enough to serve as a real baseline, through integer address mapping, primitive adjacency arrays, BFS stamping and a precomputed trader lookup table. The parallel version distributed traders across threads with per-thread BFS workspaces and a single writer thread fed through a queue. The distributed version replicated the graph across ranks and split the traders with interleaved partitioning, writing one temporary file per rank that rank 0 merged at the end.

All three modes produced identical results, and the parallel mode delivered up to a 4.2× speedup over the sequential version at depth 9. The distributed mode reached a 5.18× BFS speedup at eight ranks, with the total runtime bounded by the serial graph broadcast from rank 0, a cost that would be paid in parallel on a real cluster.

If this work were extended, the natural next steps would be running the distributed version on multiple physical machines to test true scaling, exploring graph partitioning so that ranks share the loading cost, and applying additional heuristics on top of the linkability network to flag pairs that are more likely to represent actual wash trading.